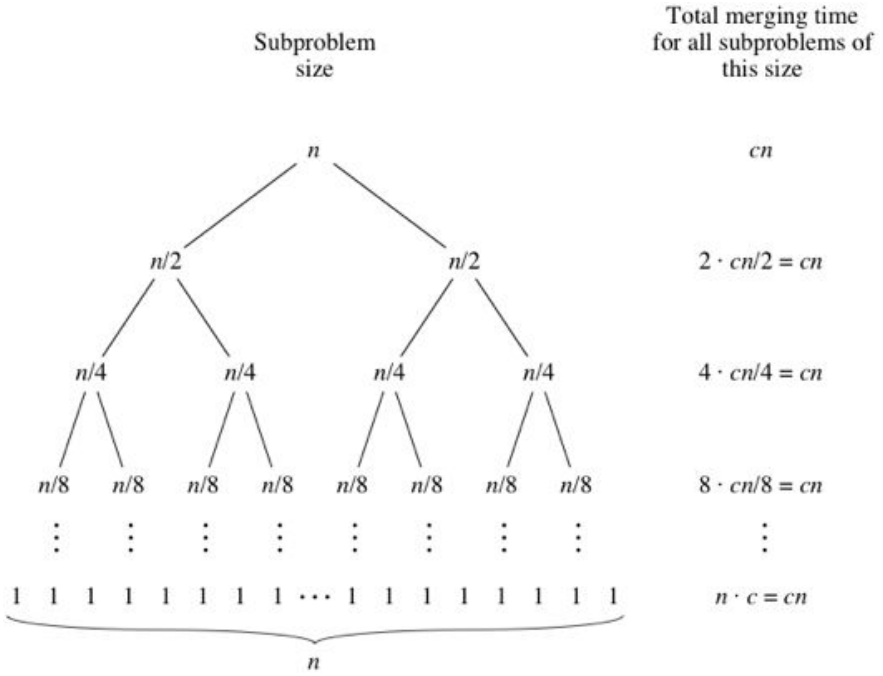



Lecture 15

# Asymptotics 3

**CS61B, Fall 2025 @ UC Berkeley**

Josh Hug and Peyrin Kao

# Toy Recursion Analysis

Lecture 15, CS61B, Fall 2025

**Recursive Runtime Analysis**

- **Toy Recursion Analysis**
- Binary Search Intuitive
- Binary Search Exact (Bonus)
- Mergesort

Asymptotic Notation

- Big O vs. Big Theta

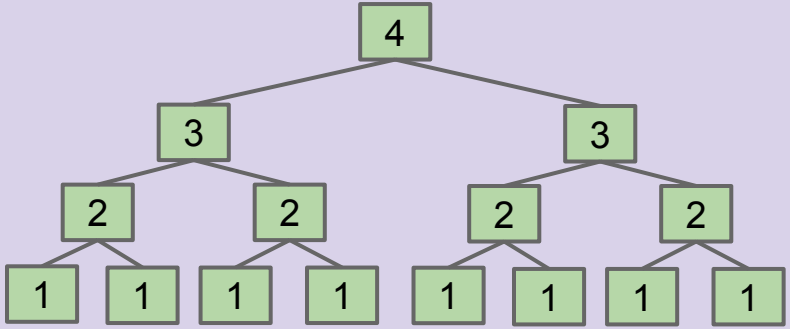
# Recursion, Approach 1: Intuitive (yellkey.com/economy)

Find a simple f(N) such that the runtime R(N) ∈ Θ(f(N)).

```
public static int f3(int n) {
   if (n <= 1)
       return 1;
   return f3(n-1) + f3(n-1);
}
```

Using your intuition, give the order of growth of the runtime of this code as a function of N?

A. 1
B. log N
C. N
D. $N^2$
E. $2^N$



# Recursion, Approach 1: Intuitive

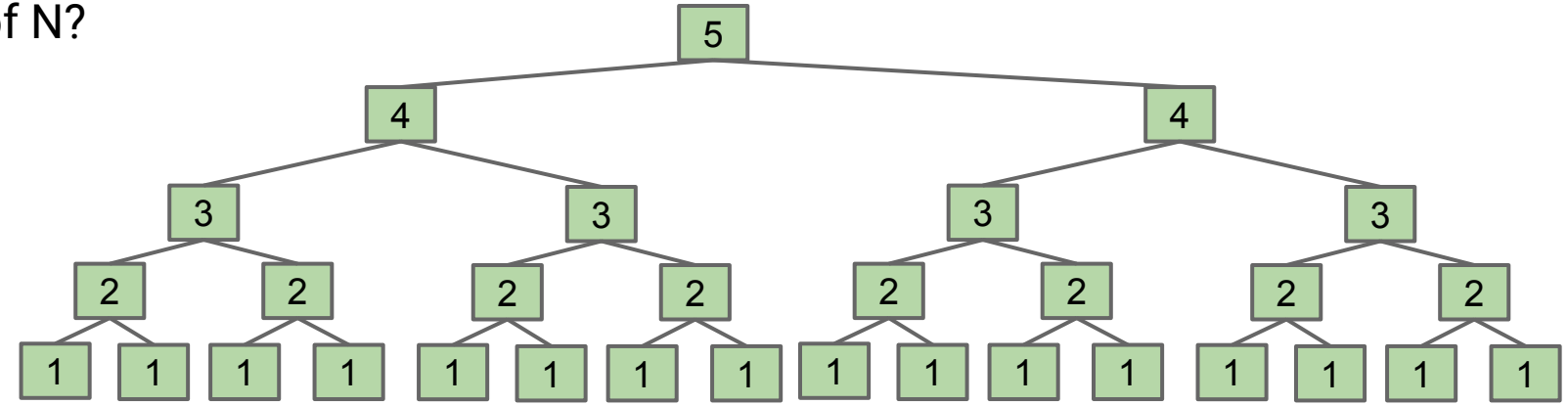
Find a simple f(N) such that the runtime R(N) ∈ Θ(f(N)).

```
public static int f3(int n) {
   if (n <= 1)
       return 1;
   return f3(n-1) + f3(n-1);
}
```

$2^N$: Every time we increase N by 1, we double the work!

Using your intuition, give the order of growth of the runtime of this code as a function of N?

A. 1
B. log N
C. N
D. $N^2$
**E. $2^N$**



# Recursion, Approach 2: Exact Counting

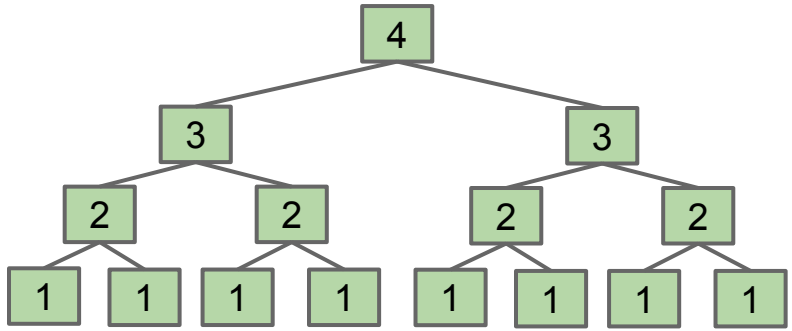
Find a simple f(N) such that the runtime R(N) ∈ Θ(f(N)).

```
public static int f3(int n) {
   if (n <= 1)
       return 1;
   return f3(n-1) + f3(n-1);
}
```

Another approach: Count number of calls to f3, given by C(N).

- C(1) = 1
- C(2) = 1 + 2
- C(3) = 1 + 2 + 4

If that's too weird, count n <= 1 operations, yields same C(N)



# Recursion, Approach 2: Exact Counting: http://yellkey.com/order

Find a simple f(N) such that the runtime R(N) ∈ Θ(f(N)).

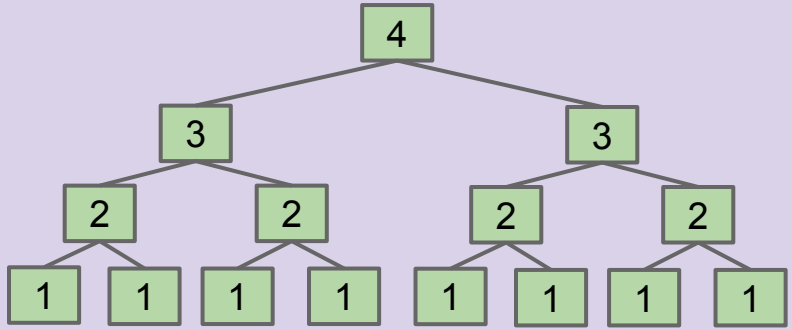
```
public static int f3(int n) {
    if (n <= 1)
        return 1;
    return f3(n-1) + f3(n-1);
}
```

Another approach: Count number of calls to f3, given by C(N).

- C(3) = 1 + 2 + 4
- C(N) = 1 + 2 + 4 + … + ???

What is the final term of the sum?

A. N
B. $2^N$
C. $2^N-1$
D. $2^{N-1}$
E. $2^{N-1}-1$



# Recursion, Approach 2: Exact Counting

Find a simple f(N) such that the runtime $R(N) \in \Theta(f(N))$.

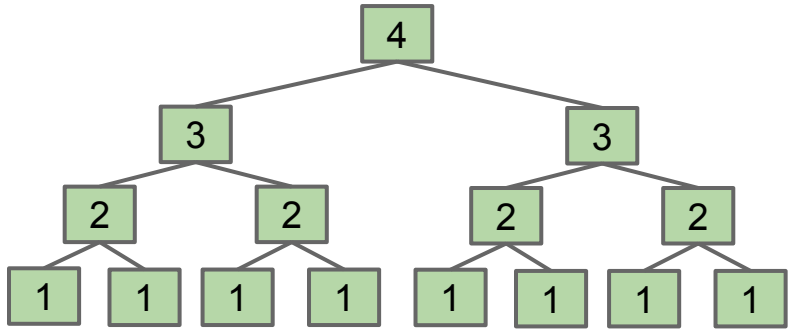
```
public static int f3(int n) {
    if (n <= 1)
        return 1;
    return f3(n-1) + f3(n-1);
}
```

Another approach: Count number of calls to f3, given by C(N).

- C(3) = 1 + 2 + 4
- C(N) = 1 + 2 + 4 + … + ???

What is the final term of the sum?

**D.** $2^{N-1}$



# Recursion, Approach 2: Exact Counting

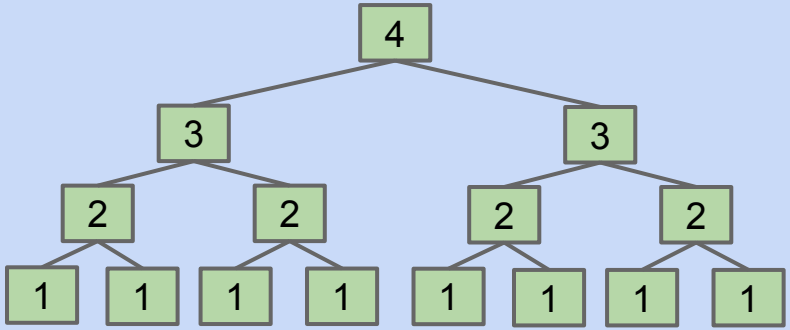
Find a simple f(N) such that the runtime $R(N) \in \Theta(f(N))$.

```
public static int f3(int n) {
    if (n <= 1)
        return 1;
    return f3(n-1) + f3(n-1);
}
```

Another approach: Count number of calls to f3, given by C(N).

- $C(N) = 1 + 2 + 4 + \ldots + 2^{N-1}$

Give a simple expression for C(N).



# Recursion, Approach 2: Exact Counting

Find a simple f(N) such that the runtime $R(N) \in \Theta(f(N))$.

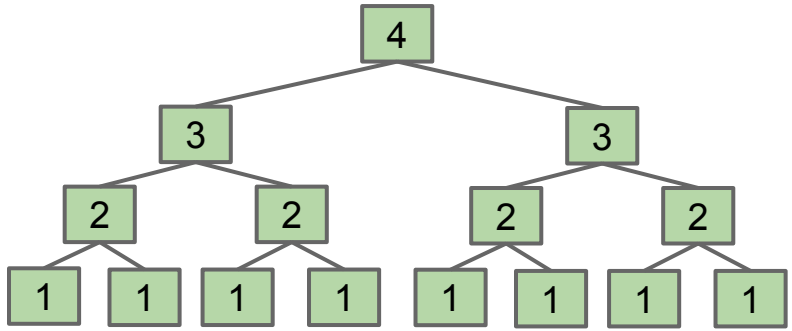
```
public static int f3(int n) {
   if (n <= 1)
       return 1;
   return f3(n-1) + f3(n-1);
}
```

Another approach: Count number of calls to f3, given by C(N).

- $C(N) = 1 + 2 + 4 + \ldots + 2^{N-1}$

Give a simple expression for C(N).

- $C(N) = 2^N - 1$
- Why? It's the Sum of First Powers of 2.
  - See next slide for details.



# Recursion, Approach 2: Exact Counting, Solving for C(N)

We want to solve: $C(N) = 1 + 2 + 4 + 8 + ... + 2^{N-1}$

We know that the Sum of the First Powers of 2 from previous lecture, i.e. as long as Q is a power of 2, then:

$$1 + 2 + 4 + 8 + ... + Q = 2Q - 1$$

Thus, since $Q = 2^{N-1}$, we have that:

$$C(N) = 2Q - 1 = 2(2^{N-1}) - 1 = 2^N - 1$$

# Recursion, Approach 2: Exact Counting

Find a simple f(N) such that the runtime $R(N) \in \Theta(f(N))$.

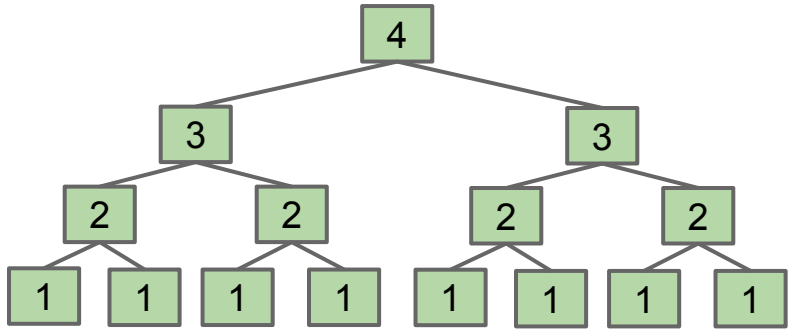
```
public static int f3(int n) {
   if (n <= 1)
       return 1;
   return f3(n-1) + f3(n-1);
}
```

Another approach: Count number of calls to f3, given by C(N).

- $C(N) = 1 + 2 + 4 + \ldots + 2^{N-1}$
- Solving, we get $C(N) = 2^N - 1$

Since work during each call is constant:

- $R(N) = \Theta(2^N)$



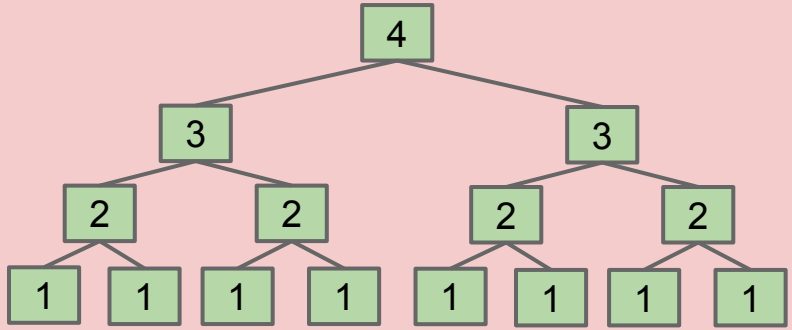
# Recursion, Approach 3: Recurrence Relations (Out of Scope for 61B)

Find a simple f(N) such that the runtime $R(N) \in \Theta(f(N))$.

```
public static int f3(int n) {
    if (n <= 1)
        return 1;
    return f3(n-1) + f3(n-1)
}
```

Third approach: Count number of calls to f3, given by a "recurrence relation" for C(N).

- C(1) = 1
- C(N) = 2C(N-1) + 1



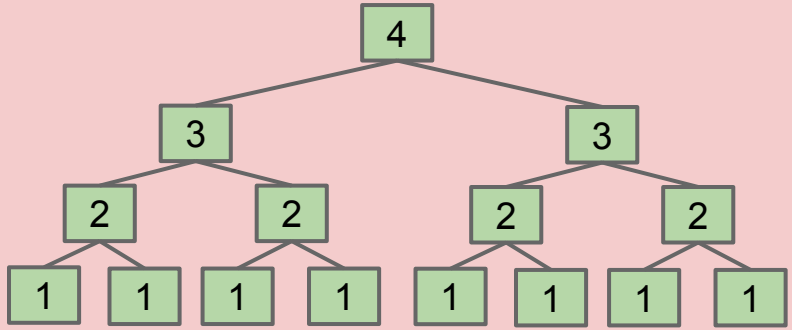
# Recursion, Approach 3: Recurrence Relations (Out of Scope for 61B)

Find a simple f(N) such that the runtime $R(N) \in \Theta(f(N))$.

```
public static int f3(int n) {
    if (n <= 1)
        return 1;
    return f3(n-1) + f3(n-1)
}
```

Third approach: Count number of calls to f3, given by a "recurrence relation" for C(N).

- C(1) = 1
- C(N) = 2C(N-1) + 1




More technical to solve. Won't do this in our course. See next slide for solution.

# Recursion, Approach 3: Recurrence Relations (Out of Scope for 61B)

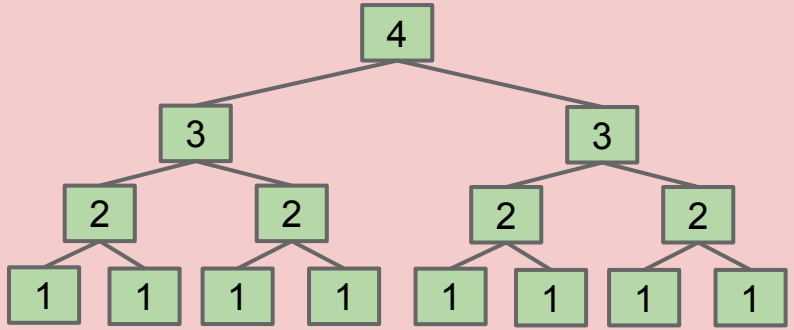
Find a simple f(N) such that the runtime R(N) ∈ Θ(f(N)).

```
public static int f3(int n) {
    if (n <= 1)
        return 1;
    return f3(n-1) + f3(n-1)
}
```

This approach not covered in class. Provided for those of you who want to see a recurrence relation solution.

Third approach: Count number of calls to f3, given by a "recurrence relation" for C(N).

$$
\begin{aligned}
C(1) &= 1 \\
C(N) &= 2C(N-1)+1 \\
&= 2(2C(N-2)+1)+1 \\
&= 2(2(2C(N-2)+1)+1)+1 \\
&= 2(\cdots 2\cdot 1+1)+1)+\cdots 1 \\
&= \underbrace{2(\cdots 2}_{N-1})\cdot 1+1)+\cdots 1 \\
&= 2^{N-1}+2^{N-2}+\cdots+1 = 2^N-1 \in \Theta(2^N)
\end{aligned}
$$



# Binary Search Intuitive

Lecture 15, CS61B, Fall 2025

**Recursive Runtime Analysis**

- Toy Recursion Analysis
- **Binary Search Intuitive**
- Binary Search Exact (Bonus)
- Mergesort

Asymptotic Notation

- Big O vs. Big Theta

# Binary Search (demo: https://goo.gl/3VvJNw)

Trivial to implement?

- Idea published in 1946.
- First correct implementation in 1962.
  - Bug in Java's binary search discovered in 2006.

See Jon Bentley's book Programming Pearls.

See this link

```
static int binarySearch(String[] sorted, String x, int lo, int hi)
    if (lo > hi) return -1;
    int m = (lo + hi) / 2;
    int cmp = x.compareTo(sorted[m]);
    if (cmp < 0) return binarySearch(sorted, x, lo, m - 1);
    else if (cmp > 0) return binarySearch(sorted, x, m + 1, hi);
    else return m;
}
```

# Binary Search (Intuitive): http://yellkey.com/share

```
static int binarySearch(String[] sorted, String x, int lo, int hi)
    if (lo > hi) return -1;
    int m = (lo + hi) / 2;
    int cmp = x.compareTo(sorted[m]);
    if (cmp < 0) return binarySearch(sorted, x, lo, m - 1);
    else if (cmp > 0) return binarySearch(sorted, x, m + 1, hi);
    else return m;
}
```

Goal: Find runtime in terms of N = hi - lo + 1 [i.e. # of items being considered]

- Intuitively, what is the order of growth of the worst case runtime?
  - A. 1
  - B. $\log_2 N$
  - C. N
  - D. $N \log_2 N$
  - E. $2^N$

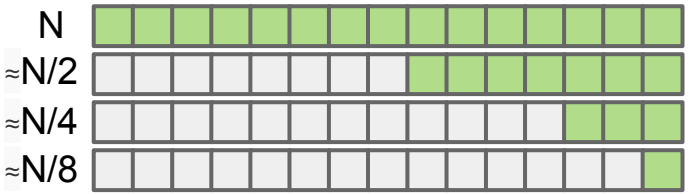
# Binary Search (Intuitive)

```
static int binarySearch(String[] sorted, String x, int lo, int hi)
    if (lo > hi) return -1;
    int m = (lo + hi) / 2;
    int cmp = x.compareTo(sorted[m]);
    if (cmp < 0) return binarySearch(sorted, x, lo, m - 1);
    else if (cmp > 0) return binarySearch(sorted, x, m + 1, hi);
    else return m;
}
```

Goal: Find runtime in terms of N = hi - lo + 1 [i.e. # of items being considered]

- Intuitively, what is the order of growth of the worst case runtime?

  **B. $\log_2 N$**




Why? Problem size halves over and over until it gets down to 1.

- If C is number of calls to binarySearch, solve for $1 = N/2^C \rightarrow C = \log_2(N)$

# Log Time Is Really Terribly Fast

In practice, logarithmic time algorithms have almost constant runtimes.

- Even for incredibly huge datasets, practically equivalent to constant time.

| N | $\log_2 N$ | Typical runtime (seconds) |
|---|---|---|
| 100 | 6.6 | 1 nanosecond |
| 100,000 | 16.6 | 2.5 nanoseconds |
| 100,000,000 | 26.5 | 4 nanoseconds |
| 100,000,000,000 | 36.5 | 5.5 nanoseconds |
| 100,000,000,000,000 | 46.5 | 7 nanoseconds |

# Binary Search Exact (Bonus)

Lecture 15, CS61B, Fall 2025

**Recursive Runtime Analysis**

- Toy Recursion Analysis
- Binary Search Intuitive
- **Binary Search Exact (Bonus)**
- Mergesort

Asymptotic Notation

- Big O vs. Big Theta

This section is available as a pre-recorded video.

I haven’t marked it as ”out of scope” since it’s just another example problem using the same techniques used throughout the lecture.

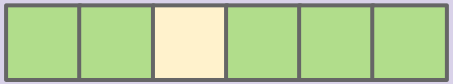
# Binary Search (Exact Count): Not a Live Video (no yellkey)

```
static int binarySearch(String[] sorted, String x, int lo, int hi)
    if (lo > hi) return -1;
    int m = (lo + hi) / 2;
    int cmp = x.compareTo(sorted[m]);
    if (cmp < 0) return binarySearch(sorted, x, lo, m - 1);
    else if (cmp > 0) return binarySearch(sorted, x, m + 1, hi);
    else return m;
}
```

Goal: Find worst case runtime in terms of N = hi - lo + 1 [i.e. # of items]

Cost model: Number of `binarySearch` calls.

N=6



- What is C(6), number of total calls for N = 6?

A. 6
B. 3
C. $\log_2(6)=2.568$
D. 2
E. 1

# Binary Search (Exact Count)

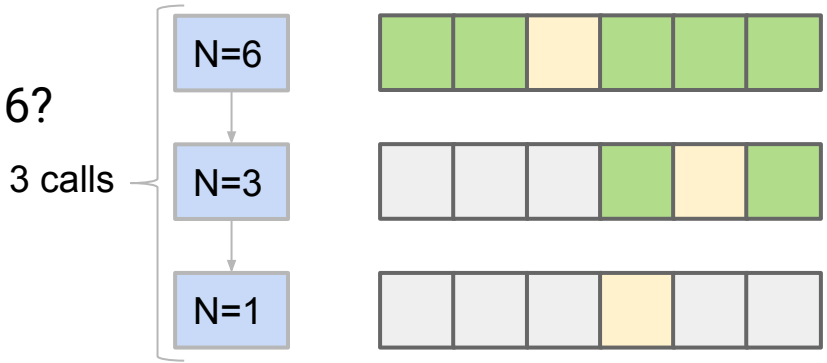
```
static int binarySearch(String[] sorted, String x, int lo, int hi)
    if (lo > hi) return -1;
    int m = (lo + hi) / 2;
    int cmp = x.compareTo(sorted[m]);
    if (cmp < 0) return binarySearch(sorted, x, lo, m - 1);
    else if (cmp > 0) return binarySearch(sorted, x, m + 1, hi);
    else return m;
}
```

Goal: Find worst case runtime in terms of N = hi - lo + 1 [i.e. # of items]

Cost model: Number of `binarySearch` calls.

- What is C(6), number of total calls for N = 6?

  **B. 3**




Three total calls, where N = 6, N = 3, and N = 1.

# Binary Search (Exact Count)

```
static int binarySearch(String[] sorted, String x, int lo, int hi)
    if (lo > hi) return -1;
    int m = (lo + hi) / 2;
    int cmp = x.compareTo(sorted[m]);
    if (cmp < 0) return binarySearch(sorted, x, lo, m - 1);
    else if (cmp > 0) return binarySearch(sorted, x, m + 1, hi);
    else return m;
}
```

Goal: Find worst case runtime in terms of N = hi - lo + 1 [i.e. # of items]

- Cost model: Number of `binarySearch` calls.

| N | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| C(N) | 1 | | | | | 3 | | | | | | | |

N=1

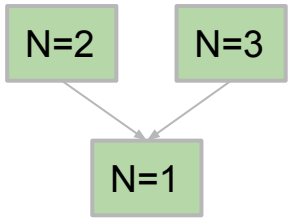
# Binary Search (Exact Count)

```
static int binarySearch(String[] sorted, String x, int lo, int hi)
    if (lo > hi) return -1;
    int m = (lo + hi) / 2;
    int cmp = x.compareTo(sorted[m]);
    if (cmp < 0) return binarySearch(sorted, x, lo, m - 1);
    else if (cmp > 0) return binarySearch(sorted, x, m + 1, hi);
    else return m;
}
```

Goal: Find worst case runtime in terms of N = hi - lo + 1 [i.e. # of items]

- Cost model: Number of `binarySearch` calls.

| N | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| C(N) | 1 | 2 | 2 | | | 3 | | | | | | | |



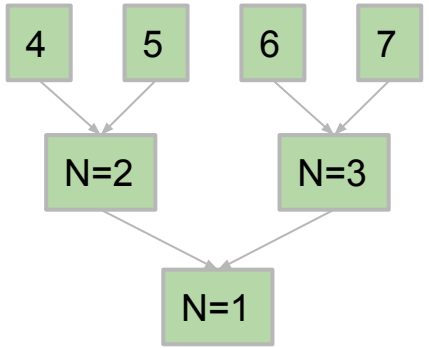
# Binary Search (Exact Count)

```
static int binarySearch(String[] sorted, String x, int lo, int hi)
    if (lo > hi) return -1;
    int m = (lo + hi) / 2;
    int cmp = x.compareTo(sorted[m]);
    if (cmp < 0) return binarySearch(sorted, x, lo, m - 1);
    else if (cmp > 0) return binarySearch(sorted, x, m + 1, hi);
    else return m;
}
```

Goal: Find worst case runtime in terms of N = hi - lo + 1 [i.e. # of items]

- Cost model: Number of `binarySearch` calls.

| N | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| C(N) | 1 | 2 | 2 | 3 | 3 | 3 | 3 | | | | | | |



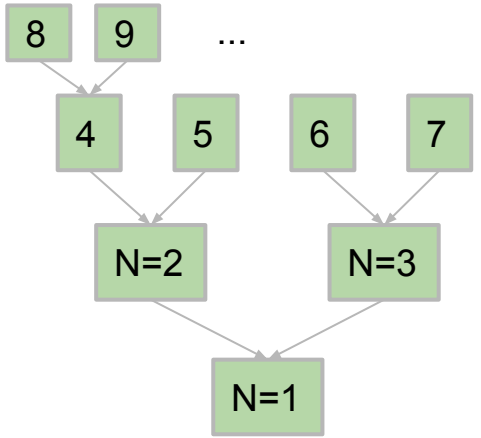
# Binary Search (Exact Count)

```
static int binarySearch(String[] sorted, String x, int lo, int hi)
    if (lo > hi) return -1;
    int m = (lo + hi) / 2;
    int cmp = x.compareTo(sorted[m]);
    if (cmp < 0) return binarySearch(sorted, x, lo, m - 1);
    else if (cmp > 0) return binarySearch(sorted, x, m + 1, hi);
    else return m;
}
```

Goal: Find worst case runtime in terms of N = hi - lo + 1 [i.e. # of items]

- Cost model: Number of `binarySearch` calls.

| N | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| C(N) | 1 | 2 | 2 | 3 | 3 | 3 | 3 | 4 | 4 | 4 | 4 | 4 | 4 |



# Binary Search (Exact Count)

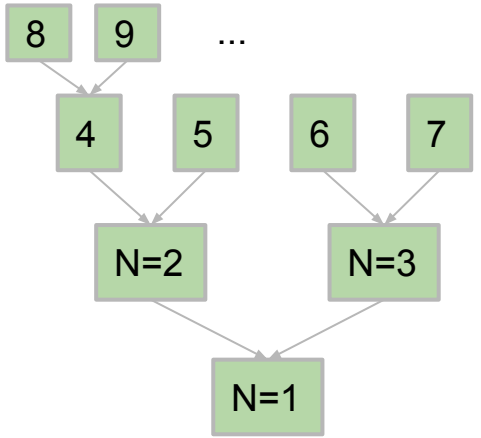
```
static int binarySearch(String[] sorted, String x, int lo, int hi)
    if (lo > hi) return -1;
    int m = (lo + hi) / 2;
    int cmp = x.compareTo(sorted[m]);
    if (cmp < 0) return binarySearch(sorted, x, lo, m - 1);
    else if (cmp > 0) return binarySearch(sorted, x, m + 1, hi);
    else return m;
}
```

Goal: Find worst case runtime in terms of N = hi - lo + 1 [i.e. # of items]

- Cost model: Number of `binarySearch` calls.

| N | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| C(N) | 1 | 2 | 2 | 3 | 3 | 3 | 3 | 4 | 4 | 4 | 4 | 4 | 4 |

$$C(N) = \lfloor \log_2(N) \rfloor + 1$$

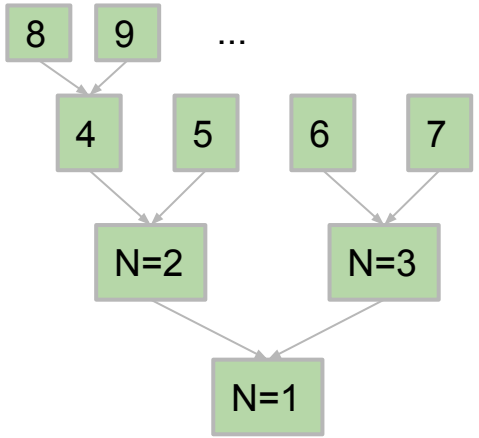


# Binary Search (Exact Count)

```
static int binarySearch(String[] sorted, String x, int lo, int hi)
    if (lo > hi) return -1;
    int m = (lo + hi) / 2;
    int cmp = x.compareTo(sorted[m]);
    if (cmp < 0) return binarySearch(sorted, x, lo, m - 1);
    else if (cmp > 0) return binarySearch(sorted, x, m + 1, hi);
    else return m;
}
```

Goal: Find worst case runtime in terms of N = hi - lo + 1 [i.e. # of items]

- Cost model: Number of `binarySearch` calls.
- $C(N) = \lfloor \log_2(N) \rfloor + 1$
- Since each call takes constant time, $R(N) = \Theta(\lfloor \log_2(N) \rfloor)$
  - This f(N) is way too complicated. Let's simplify.



# Handy Big Theta Properties

Goal: Simplify $\Theta(\lfloor \log_2(N) \rfloor)$

For proof:
See skipped slide.

- Three handy properties to help us simplify:
  - $\lfloor f(N) \rfloor = \Theta(f(N))$ [the floor of f has same order of growth as f]
  - $\lceil f(N) \rceil = \Theta(f(N))$ [the ceiling of f has same order of growth as f]
  - $\log_P(N) = \Theta(\log_Q(N))$ [logarithm base does not affect order of growth]

$\lfloor \log_2(N) \rfloor = \Theta(\log N)$

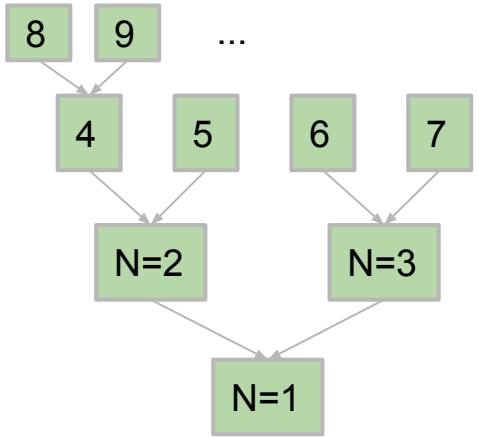
Since base is irrelevant, we omit from our big theta expression. We also omit the parenthesis around N for aesthetic reasons.

# Binary Search (Exact Count)

```
static int binarySearch(String[] sorted, String x, int lo, int hi)
    if (lo > hi) return -1;
    int m = (lo + hi) / 2;
    int cmp = x.compareTo(sorted[m]);
    if (cmp < 0) return binarySearch(sorted, x, lo, m - 1);
    else if (cmp > 0) return binarySearch(sorted, x, m + 1, hi);
    else return m;
}
```

Goal: Find worst case runtime in terms of N = hi - lo + 1 [i.e. # of items]

- Cost model: Number of `binarySearch` calls.
- $C(N) = \lfloor \log_2(N) \rfloor + 1 = \Theta(\log N)$
- Since each call takes constant time, $R(N) = \Theta(\log N)$




... and we're done!

# Binary Search (using Recurrence Relations)

```
static int binarySearch(String[] sorted, String x, int lo, int hi)
    if (lo > hi) return -1;
    int m = (lo + hi) / 2;
    int cmp = x.compareTo(sorted[m]);
    if (cmp < 0) return binarySearch(sorted, x, lo, m - 1);
    else if (cmp > 0) return binarySearch(sorted, x, m + 1, hi);
    else return m;
}
```

Approach: Measure number of string comparisons for N = hi - lo + 1.

- C(0) = 0
- C(1) = 1
- C(N) = 1 + C((N-1)/2)

Can show that C(N) = Θ(log N). Beyond scope of class, so won't solve in slides.

# Mergesort

Lecture 15, CS61B, Fall 2025

**Recursive Runtime Analysis**

- Toy Recursion Analysis
- Binary Search Intuitive
- Binary Search Exact (Bonus)
- **Mergesort**

Asymptotic Notation

- Big O vs. Big Theta

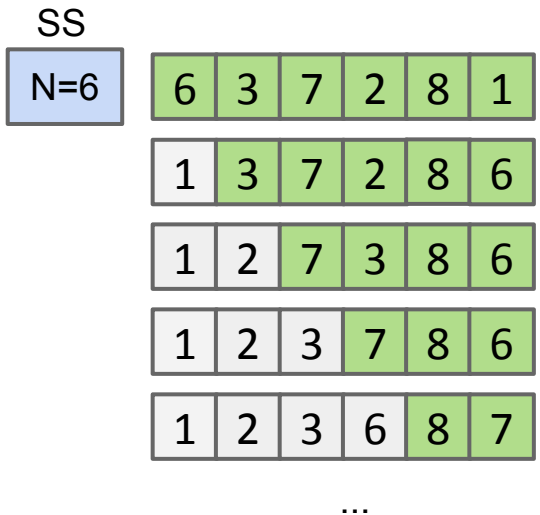
# Selection Sort: A Prelude to Mergesort

Earlier in class we discussed a sort called selection sort:

- Find the smallest unfixed item, move it to the front, and 'fix' it.
- Sort the remaining unfixed items using selection sort.

Runtime of selection sort is $\Theta(N^2)$:

- Look at all N unfixed items to find smallest.
- Then look at N-1 remaining unfixed.
- …
- Look at last two unfixed items.
- Done, sum is $2+3+4+5+...+N = \Theta(N^2)$



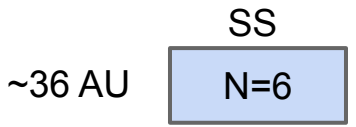
# Selection Sort: A Prelude to Mergesort

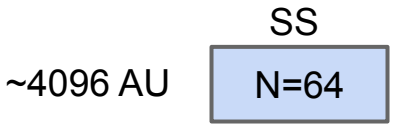
Earlier in class we discussed a sort called selection sort:

- Find the smallest unfixed item, move it to the front, and 'fix' it.
- Sort the remaining unfixed items using selection sort.

Runtime of selection sort is $\Theta(N^2)$:

- Look at all N unfixed items to find smallest.
- Then look at N-1 remaining unfixed.
- …
- Look at last two unfixed items.
- Done, sum is $2+3+4+5+...+N = \Theta(N^2)$



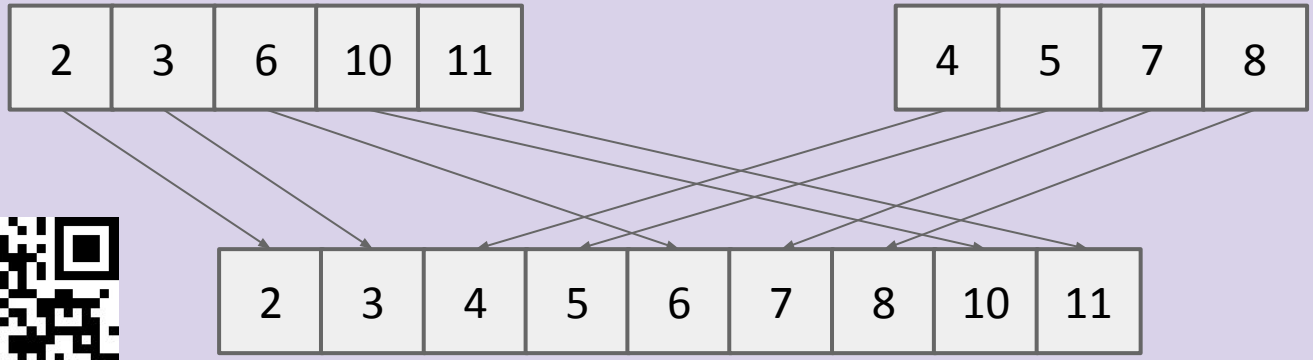

Given that runtime is quadratic, for N = 64, we might say the runtime for selection sort is 4,096 arbitrary units of time (AU).

# The Merge Operation: Another Prelude to Mergesort

Given two sorted arrays, the merge operation combines them into a single sorted array by successively copying the smallest item from the two arrays into a target array.

Merging Demo (Link)

# Merge Runtime: http://yellkey.com/reduce




How does the runtime of merge grow with N, the total number of items?

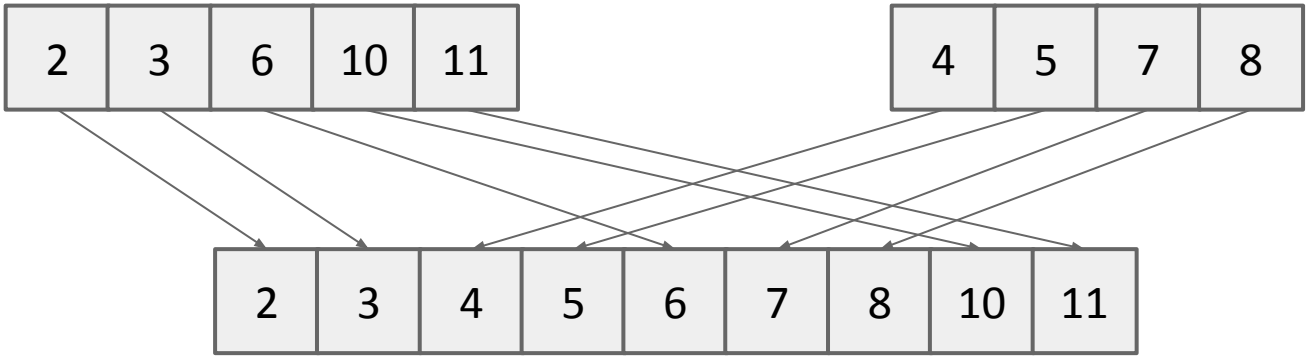
A. $\Theta(1)$

B. $\Theta(\log N)$

C. $\Theta(N)$

D. $\Theta(N^2)$

# Merge Runtime



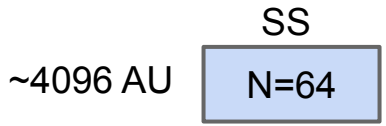

How does the runtime of merge grow with N, the total number of items?

**C. Θ(N)**. Why? Use array writes as cost model, merge does exactly N writes.

# Using Merge to Speed Up the Sorting Process

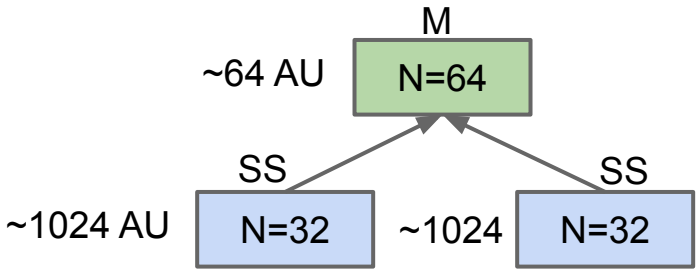
Merging can give us an improvement over vanilla selection sort:

- Selection sort the left half: $\Theta(N^2)$.
- Selection sort the right half: $\Theta(N^2)$.
- Merge the results: $\Theta(N)$.




N=64: ~2112 AU.

- Merge: ~64 AU.
- Selection sort: ~2*1024 = ~2048 AU.




Still $\Theta(N^2)$, but faster since $N+2*(N/2)^2 < N^2$

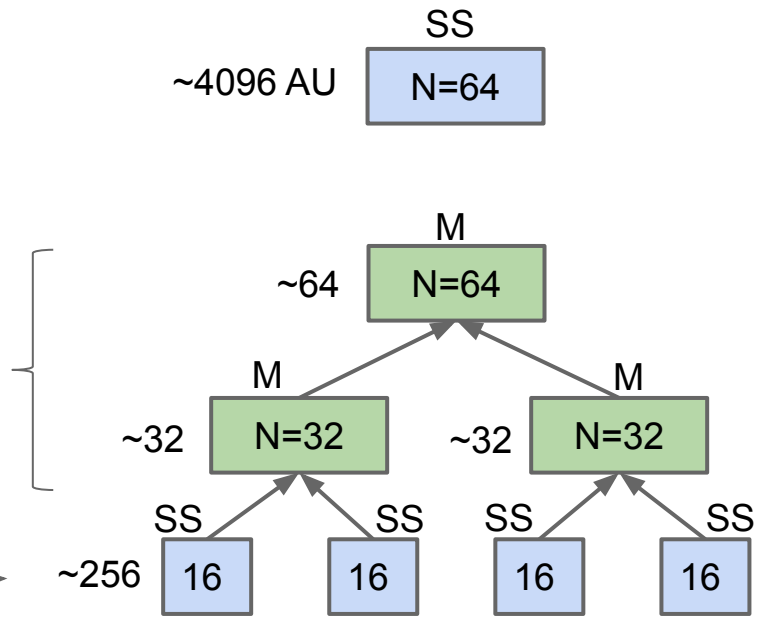
- ~2112 vs. ~4096 AU for N=64.

# Two Merge Layers

Can do even better by adding a second layer of merges.

Runtime for each sort:

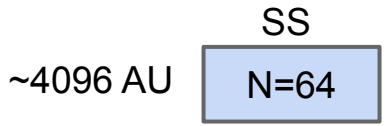
- Selection sort only: ~4096 AU.
- One layer of merges: ~2112 AU.
- Two layers of merges:  ~1152 AU.
  - Merge: ~64 AU + 2*~32 AU.
  - Selection sort: 4*~256.



# Example: Mergesort

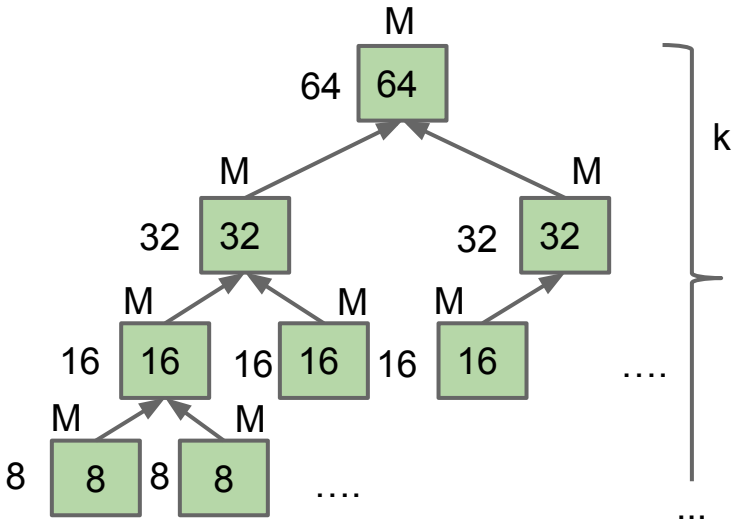
Mergesort does merges all the way down (no selection sort):

- If array is of size 1, return.
- Mergesort the left half: Θ(??).
- Mergesort the right half: Θ(??).
- Merge the results: Θ(N).




Total runtime to merge all the way down: ~384 AU

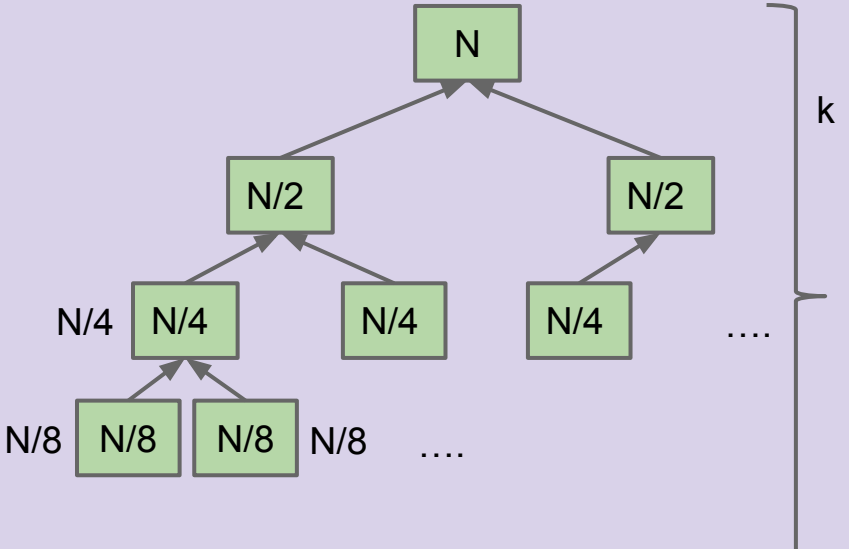
- Top layer: ~64 = 64 AU
- Second layer: ~32*2 = 64 AU
- Third layer: ~16*4 = 64 AU
- Overall runtime in AU is ~64k, where k is the number of layers.
- k = $\log_2(64) = 6$, so ~384 total AU.



# Example: Mergesort Order of Growth, yellkey.com/clear

For an array of size N, what is the worst case runtime of Mergesort?

A. Θ(1)
B. Θ(log N)
C. Θ(N)
D. Θ(N log N)
E. Θ($N^2$)



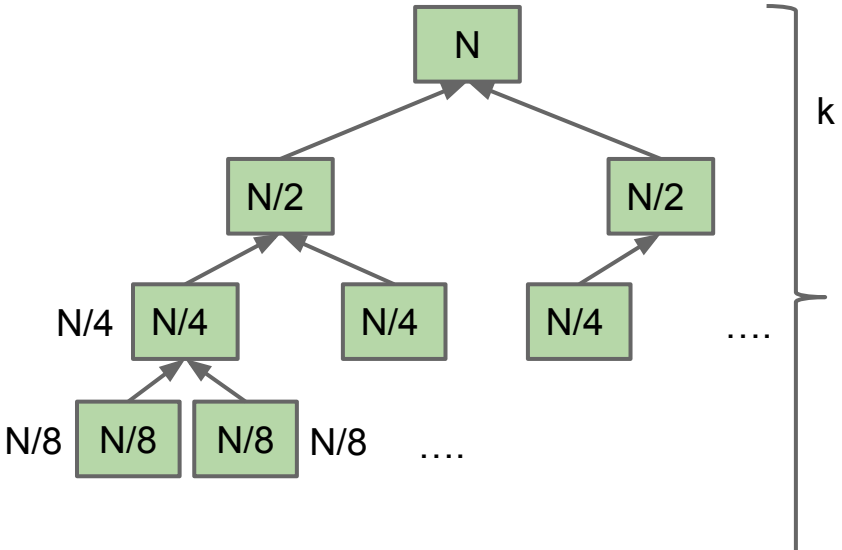
# Example: Mergesort Order of Growth

Mergesort has worst case runtime = Θ(N log N).

- Every level takes ~N AU.
  - Top level takes ~N AU.
  - Next level takes ~N/2 + ~N/2 = ~N.
  - One more level down: ~N/4 + ~N/4 + ~N/4 + ~N/4 = ~N.
- Thus, total runtime is ~Nk, where k is the number of levels.
  - How many levels? Goes until we get to size 1.
  - k = $\log_2(N)$.
- Overall runtime is Θ(N log N).

Exact count explanation is tedious.

- Omitted here. See textbook.



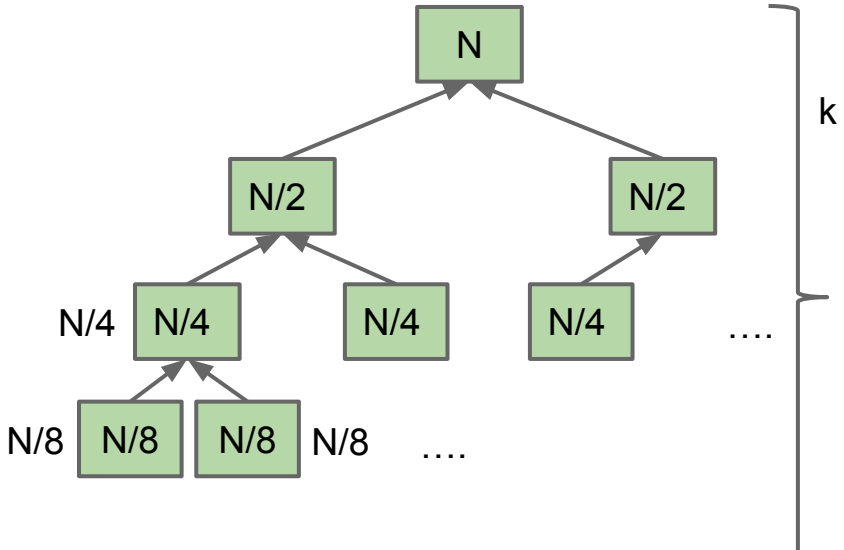
# Mergesort using Recurrence Relations (Extra)

C(N): Number of calls to mergesort + number of array writes.

$$C(N) = \begin{cases} 1 & : N < 2 \\ 2C(N/2) + N & : N \geq 2 \end{cases}$$

$$\begin{aligned} C(N) &= 2(2C(N/4) + N/2) + N \\ &= 4C(N/4) + N + N \\ &= 8C(N/8) + N + N + N \\ &= N \cdot 1 + \underbrace{N + N + \ldots + N}_{k = \lg N} \\ &= N + N \lg N \in \Theta(N \lg N) \end{aligned}$$

Only works for $N=2^k$. Can be generalized at the expense of some tedium by separately finding Big O and Big Omega bounds.



# Linear vs. Linearithmic (N log N) vs. Quadratic

N log N is basically as good as N, and is vastly better than $N^2$.

- For N = 1,000,000, the log N is only 20.

| | $n$ | $n \log_2 n$ | $n^2$ | $n^3$ | $1.5^n$ | $2^n$ | $n!$ |
|---|---|---|---|---|---|---|---|
| $n = 10$ | < 1 sec | < 1 sec | < 1 sec | < 1 sec | < 1 sec | < 1 sec | 4 sec |
| $n = 30$ | < 1 sec | < 1 sec | < 1 sec | < 1 sec | < 1 sec | 18 min | $10^{25}$ years |
| $n = 50$ | < 1 sec | < 1 sec | < 1 sec | < 1 sec | 11 min | 36 years | very long |
| $n = 100$ | < 1 sec | < 1 sec | < 1 sec | 1 sec | 12,892 years | $10^{17}$ years | very long |
| $n = 1{,}000$ | < 1 sec | < 1 sec | 1 sec | 18 min | very long | very long | very long |
| $n = 10{,}000$ | < 1 sec | < 1 sec | 2 min | 12 days | very long | very long | very long |
| $n = 100{,}000$ | < 1 sec | 2 sec | 3 hours | 32 years | very long | very long | very long |
| $n = 1{,}000{,}000$ | 1 sec | 20 sec | 12 days | 31,710 years | very long | very long | very long |

**Table 2.1** The running times (rounded up) of different algorithms on inputs of increasing size, for a processor performing a million high-level instructions per second. In cases where the running time exceeds $10^{25}$ years, we simply record the algorithm as taking a very long time.

(from Algorithm Design: Tardos, Kleinberg)

# Big O vs. Big Theta

Lecture 15, CS61B, Fall 2025

Recursive Runtime Analysis

- Toy Recursion Analysis
- Binary Search Intuitive
- Binary Search Exact (Bonus)
- Mergesort

**Asymptotic Notation**

- **Big O vs. Big Theta**

# Big Theta vs. Big O

We saw this in the last lecture! But let's review this subtle topic. If we have big theta, why do we also have big O?

The answer is the same as: If we have $=$, why do we also have $\leq$.

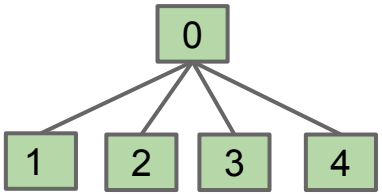
# BST Tree Height

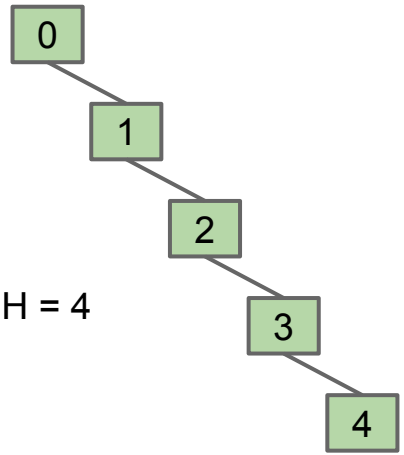
Let's start today by carefully discussing the height of Quick Union Trees.

A quick union tree of N connected items ranges in height from 1 to N - 1.

- Difference is dramatic! Some trees are "flat", others are "spindly".

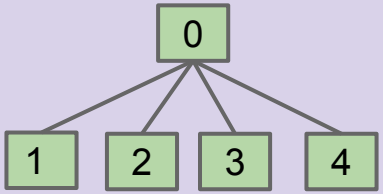
N=5, H = 1

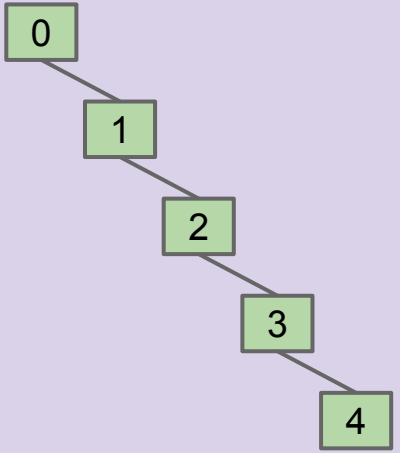



N=5, H = 4

# Tree Height: http://yellkey.com/food



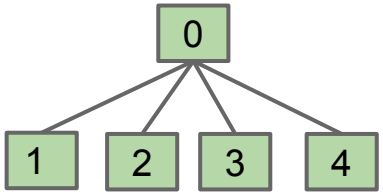




Let H(N) be the height of a tree with N nodes. Give H(N) in Big-Theta notation for "flat" and "spindly" trees, respectively:

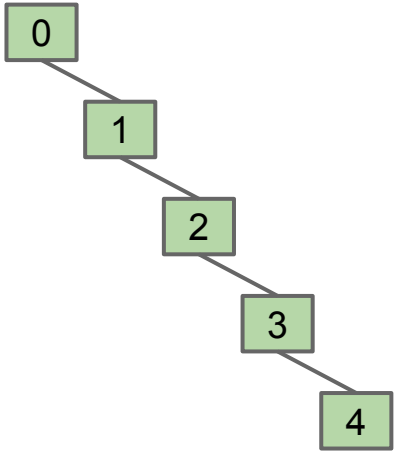
A. Θ(1), Θ(1)
B. Θ(1), Θ(N)
C. Θ(N), Θ(1)
D. Θ(N), Θ(N)

# Tree Height

Height varies dramatically between “flat” and “spindly” trees.




H = Θ(1)




H = Θ(N)

Performance of operations on quick union can be very bad.

- Example: isConnected(0, N-1) would take linear time on spindly tree.

# Statements about Tree Height (no yellkey)

Which of these statements are true?

A. Worst case Quick Union tree height is $\Theta(N)$.

B. Quick Union tree height is $O(N)$.

C. Quick Union tree height is $O(N^2)$.

# Statements about Tree Height

Which of these statements are true?

**A. Worst case Quick Union tree height is Θ(N). IS EQUAL TO LINEAR**

**B. Quick Union tree height is O(N). IS LESS THAN OR EQUAL TO LINEAR**

**C. Quick Union tree height is $O(N^2)$. IS LESS THAN OR EQUAL TO QUADRATIC**

All are **true**!

- A worst case (spindly tree) WQU tree has a height that grows exactly linearly - Θ(N).
- All WQU trees have a height that is linear or better - O(N).
- All WQUs trees have a height that that is quadratic or better - $O(N^2)$.

# Statements about Tree Height (no yellkey)

Which of these statements is more informative?

A. Worst case Quick Union tree height is $\Theta(N)$.

B. Quick Union tree height is $O(N)$.

C. Quick Union tree height is $O(N^2)$.

# Statements about Tree Height

Which of these statements is more informative?

**A. Worst case Quick Union tree height is Θ(N).**

B. Quick Union tree height is O(N).

C. Quick Union tree height is $O(N^2)$.

Saying that the worst case has order of growth N is more informative than saying the height is O(N).

Let's see an analogy.

# Question (no yellkey)

Which statement gives you more information about a hotel?

A. The most expensive room in the hotel is $639 per night.

B. Every room in the hotel is less than or equal to $639 per night.

# Question

Which statement gives you more information about a hotel?

**A. The most expensive room in the hotel is $639 per night.**

B. Every room in the hotel is less than or equal to $639 per night.

Most expensive room: $639/nt

THE RITZ-CARLTON
LAKE TAHOE

All rooms <= $639/nt

THE RITZ-CARLTON
LAKE TAHOE


(A nice place to stay!)

# BST Height

Quick Union Tree height is all four of these:

- O(N).
- Θ(1) in the best case (“flat”).
- Θ(N) in the worst case (“spindly”).
- $O(N^2)$.

The middle two statements are more informative.

- Big O is NOT mathematically the same thing as “worst case”.
  - … but Big O often used as shorthand for “worst case”.

Big O is very often (and incorrectly) used an exact synonym for worst case.

- But it’s more precise to say “Θ(N) in the worst case” rather than “O(N)”.

# The Usefulness of Big O

Big O is still a useful idea:

- Allows us to make simple blanket statements, e.g. can just say “binary search is O(log N)” instead of “binary search is Θ(log N) in the worst case”.
- Sometimes don’t know the exact runtime, so use O to give an upper bound.
  - Example: Runtime for finding shortest route that goes to all world cities is $O(2^N)^*$. There might be a faster way, but nobody knows one yet.
- Easier to write proofs for Big O than Big Theta, e.g. finding runtime of mergesort, you can round up the number of items to the next power of 2. A little beyond the scope of our course.
- If you can show that a function is $O(N^2)$ and also $\Omega(N^2)$, then you know that it is also Θ(N^2).
  - Just like if you show a <= b and b <= a, you know that a = b.

*: Under certain assumptions and constraints not listed.

# Summary

We already said this last time, but asymptotic analysis requires **careful thought**.

- There are **no magic shortcuts** for analyzing code.
- In our course, it's OK to do exact counting or intuitive analysis.
  - Know how to sum 1 + 2 + 3 … + N and 1 + 2 + 4 + … + N.
  - We won't be writing mathematical proofs in this class.
- Many runtime problems you'll do in this class resemble one of the five problems from the last three lectures. See textbook, discussions, old exams for more practice.

Different solutions to the same problem, e.g. sorting, may have different runtimes.

- $N^2$ vs. $N \log N$ is an enormous difference.
- Going from $N \log N$ to $N$ is nice, but not a radical change.

# Image Credit

Image of Mergesort runtime on title slide from https://www.khanacademy.org/computing/computer-science/algorithms/merge-sort/a/analysis-of-merge-sort

# Attendance

yellkey.com/total

Sp26 TODO: Add a two variable asymptotics example.